

J.P. MORGAN PAYMENT GATEWAY TESTING GUIDE FOR SALESFORCE COMMERCE CLOUD (SFRA) CARTRIDGE

Testing Guide

DOCUMENT CONTROL

Document Title	J.P. MORGAN PAYMENT GATEWAY TESTING GUIDE FOR SALESFORCE COMMERCE CLOUD (SFRA) CARTRIDGE
Document Version	1.0
Document Status	Draft
Last Updated (YYYY-MM-DD)	21-May-2026
Cartridge Version	26.1.0
Compatibility	SFRA (Storefront Reference Architecture)
Supported Cards	Visa, Mastercard, American Express, Discover, Diners, JCB etc
Supported Currency	USD, CAD
Supported Locale	en_US. en_CA

REVISION HISTORY

VERSION	DATE	AUTHOR	SUMMARY OF CHANGES
1.0	21-May-2026	JPMC	Initial release

TABLE OF CONTENTS

Document Control	ii
REVISION HISTORY	iii
1. Introduction	1
1.1 Summary.....	1
1.2 Test Cases	1
1.3 Test Data.....	1
2. Automated Test Cases (Playwright & TypeScript).....	2
2.1 Prerequisites & Configuration.....	2
2.2 Initial Setup	3
2.3 Configuration	3
3. NPM Scripts Reference	6
3.1 Run Scripts	6
3.2 Running Individual Tests.....	7
3.3 What Each File Does.....	8
4. Test Suites.....	10
4.1 Test Specs	10
4.2 Writing New Tests.....	11
4.3 Current Baseline	12
5. CI/CD Integration	13

5.1 Set these environment variables in your pipeline:	13
5.2 Pipeline steps:	13
6. Troubleshooting	14
7. Viewing Results	15
8. Full Backend Test Suite	16
8.1 Unit Tests	16
8.2 Integration Tests	16

1. INTRODUCTION

1.1 Summary

This document serves as a comprehensive testing guide for the J.P. Morgan Chase (JPMC) Payment Gateway integration within the Salesforce Reference Architecture (SFRA) environment. The purpose of this guide is to provide Quality Assurance (QA) and development teams with standardized test cases, configuration prerequisites, and troubleshooting steps to ensure seamless checkout experience.

1.2 Test Cases

This cartridge has successfully passed through QA. The test cases below help you to implement the functional and integration test cases. Refer

[link_jpmc/test/JPMC_TestCases_Suite.xlsx](#)

1.3 Test Data

The test data can be found on [JPMC TESTING DOC](#)

2. AUTOMATED TEST CASES (PLAYWRIGHT & TYPESCRIPT)

To ensure continuous quality and prevent regressions, the following test cases have been automated using Playwright and TypeScript. These scripts validate critical user journeys, consent handling, and end-to-end checkout functionality

- *Checkout - Guest User Payment*
- *Checkout - Registered User with Credit Card and Other payments*
- *Checkout - Saved Payment Method*
- *Confirmation Page Validation*
- *One Error Scenario*
- *Account Payment Validation*

Note : Tested on Chrome Browser

2.1 Prerequisites & Configuration

System Requirements

- *Node.js v16 or higher*
- *JPMC LINK cartridge deployed and configured on your SFCC sandbox*
- *A registered test user on the storefront (for logged-in checkout and account tests)*
- *SFRA RefArch site running with the JPMC cartridge in the cartridge path*

2.2 Initial Setup

All commands are run from the `link_jpmc/` root directory.

- *Step 1 — Install dependencies and browser*
`npm run test:functional:setup`
This runs `npm install` inside `test/functional/` and downloads a sandboxed Chromium binary (~170 MB) into `~/Library/Caches/ms-playwright/` (macOS) or `~/.cache/ms-playwright/` (Linux).
It does not install Chrome system-wide.
- *Step 2 — Create your environment file*
`cp test/functional/.env.example test/functional/.env`
- *Step 3 — Fill in your sandbox values*
Edit `test/functional/.env` — see Configuration below.

2.3 Configuration

All runtime settings live in **`test/functional/.env`** (git-ignored). The committed **`.env.example`** serves as the template.

Required Variables

Variable	Description	Example
TEST_ENV	Which environment prefix to use	dev
DEV_BASE_URL	Sandbox base URL	https://abcd-001.dx.commercecloud.salesforce.com
DEV_LOGIN_URL	Full Login-Show URL for your site	https://abcd-001.dx.commercecloud.salesforce.

		com/on/demandware.store/Sites-RefArch-Site/en_US/Login-Show
DEV_HOME_URL	Storefront home URL	https://abcd-001.dx.commercecloud.salesforce.com/on/demandware.store/Sites-RefArch-Site/en_US/
DEV_TEST_USER_EMAIL	Registered storefront account email	testuser@example.com
DEV_TEST_USER_PASSWORD	Password for that account	(your password)

Replace **DEV_** with **STAGING_** or **PROD_** and set **TEST_ENV** accordingly to target other environments.

Optional Variables

Variable	Default	Description
HEADLESS	false	true = run browser without UI (for CI)
DEFAULT_TIMEOUT	30000	Global test timeout in ms
NAVIGATION_TIMEOUT	60000	Page navigation timeout in ms
ACTION_TIMEOUT	15000	Click/fill/select action timeout in ms

EXPECT_TIMEOUT	15000	Assertion timeout in ms
WAIT_SHORT	500	Pause between form field fills (ms)
WAIT_MEDIUM	1500	Post-click settle wait (ms)
WAIT_LONG	3000	Wait for dynamic page state changes (ms)
CI	false	Set true in pipelines (enables retries, serial execution)

3. NPM SCRIPTS REFERENCE

3.1 Run Scripts

Run all scripts from the **link_jpmc/** root directory.

Script	Command	What It Does
test:functional:setup	npm run test:functional:setup	Installs test dependencies (test/functional/node_modules) + downloads Chromium browser binary. Run once after cloning or after dependency changes.
test:functional	npm run test:functional	Runs all functional tests in headless/headed mode (controlled by HEADLESS in .env). Executes in parallel with 4 workers.
test:functional:headed	npm run test:functional:headed	Runs all tests with a visible browser window regardless of HEADLESS setting. Useful for watching test execution.
test:functional:ui	npm run test:functional:ui	Opens Playwright's interactive UI mode — a visual test runner where you can pick tests, watch them execute step-by-step, inspect DOM, view traces. Best for debugging.

test:functional:report	npm run test:functional:report	Opens the HTML test report from the last run in your default browser. Shows pass/fail per test, screenshots on failure, and execution traces.
------------------------	-----------------------------------	--

3.2 Running Individual Tests

Run a specific test file

- *cd test/functional && npx playwright test tests/guest-checkout.spec.ts*

Run tests matching a name pattern

- *cd test/functional && npx playwright test --grep "Guest Checkout"*

Debug a specific test (pauses on each step)

- *cd test/functional && npx playwright test tests/login.spec.ts --debug*

3.3 What Each File Does

File	Purpose
config/environment.ts	Reads .env using .env, validates required variables exist, exports a typed envConfig object used by all other files.
playwright.config.ts	Configures Playwright: test directory, parallelism, retries, timeouts, browser (Chromium), screenshot-on-failure. All values driven by envConfig.
utils/fixtures.ts	Extends Playwright's test with custom fixtures so page objects are automatically instantiated and injected into every test function.
utils/helpers.ts	Three utilities: logStep() for console output, takeScreenshot() with timeout/fallback behavior, generateRandomEmail() for guest checkout.
test-data/testData.ts	Single source of truth for test data: card fixtures, sanitized owner/shipping names, user credentials (from env), product info.
pages/BasePage.ts	Abstract base with timeouts and waits from env, plus verifyUrlContains() and verifyElementVisible() used by all page objects.

pages/CheckoutPage.ts	Largest page object — handles guest email entry, shipping form fill, payment form, saved payment CVV, place order, and all verification methods.
-----------------------	--

4. TEST SUITES

4.1 Test Specs

- *Login (login.spec.ts)*
 - *Valid login — Navigate → consent popup → enter credentials → verify dashboard*
 - *Invalid login — Enter wrong credentials → verify stays on login page*
- *Guest Checkout (guest-checkout.spec.ts)*
 - *Full happy path:*
 - *Home → Category (New Arrivals > Mens) → PDP (Modern Sport Coat) → Select Size → Add to Cart → Mini-Cart → Guest Email → Shipping Form → Gift Options → Payment Form → Review → Place Order → Order Confirmation*
- *Registered User Checkout (checkout-e2e.spec.ts)*
- *Two scenarios:*
 - *New card — Login → Browse → Cart → Checkout → Add new card → Place order → Confirmation*
 - *Saved card — Login → Browse → Cart → Checkout → Verify payment methods (Credit, Google Pay) → Enter CVV for saved card → Place order → Confirmation*
- *Checkout Negative Path (checkout-negative.spec.ts)*
- *Same guest checkout flow but asserts that a payment error message is displayed after placing the order (validates error handling for declined/failed payments).*
- *Account Payment Management (account-payment-validation.spec.ts)*

- *Add single card — Login → Dashboard → Add Payment → Fill form → Verify on wallet page*
- *Verify form labels — All payment form labels present and correct*
- *Add multiple cards — Visa + Mastercard + Amex sequentially*
- *Delete card — Add card → Delete → Verify removal*

4.2 Writing New Tests

Pattern

```
import { test, expect } from '../utils/fixtures';
import { TestData } from '../test-data/testData';
import { logStep } from '../utils/helpers';
test.describe('My Feature', () => {
  test('should do something', async ({ loginPage, checkoutPage }) => {
    logStep('Login');
    await loginPage.performLogin(
      TestData.urls.loginPage,
      TestData.users.validUser.email,
      TestData.users.validUser.password
    );
    logStep('Do action');
  });
});
```


Key Points

- *Import test from ../utils/fixtures (not @playwright/test) to get page object fixtures*
- *Use logStep() for readable console output during execution*
- *All test data comes from TestData — don't hardcode values*
- *Page objects handle all locator logic — tests describe the what, pages handle the how*
- *Add new page objects by creating a class extending BasePage, then register in utils/fixtures.ts*

4.3 Current Baseline

- *11 tests across 5 spec files*
- *Chromium-only project configuration*
- *Screenshot capture on failure in Playwright config, plus manual helper support*
- *Guest emails generated in yopmail format via generateRandomEmail()*

5. CI/CD INTEGRATION

5.1 Set these environment variables in your pipeline:

- *env:*
- *TEST_ENV: dev*
- *DEV_BASE_URL: \${ secrets.SFCC_SANDBOX_URL }*
- *DEV_LOGIN_URL: \${ secrets.SFCC_LOGIN_URL }*
- *DEV_HOME_URL: \${ secrets.SFCC_HOME_URL }*
- *DEV_TEST_USER_EMAIL: \${ secrets.TEST_USER_EMAIL }*
- *DEV_TEST_USER_PASSWORD:*
\${ secrets.TEST_USER_PASSWORD }
- *HEADLESS: true*
- *CI: true*

5.2 Pipeline steps:

- *npm run test:functional:setup*
- *npm run test:functional*
- *When CI=true:*
 - *Tests run serially (1 worker) for stability*
 - *Failed tests are retried 2 times automatically*
 - *Screenshots captured on every failure*

6. TROUBLESHOOTING

Problem	Cause	Fix
browserType.launch: Executable doesn't exist	Chromium not installed	Run <code>npm run test:functional:setup</code>
Tests time out on navigation	Sandbox is down or URLs wrong	Verify <code>DEV_BASE_URL</code> points to a live instance
Login tests fail with URL still on /Login- Show	Invalid credentials or account locked	Verify email/password in <code>.env</code> can login manually
Consent modal blocks clicks	Popup reappears on new pages	<code>handleConsentPopup()</code> is called in tests — check if your site has a different modal selector
Mini-cart checkout button not found	SFRA template customization changed classes	Update locator in <code>NavigationPage.ts</code>
Tests pass locally, fail in CI	Browser needs headless mode	Set <code>HEADLESS=true</code> and <code>CI=true</code>
Missing required env vars error	<code>.env</code> file not created or incomplete	Run <code>cp .env.example .env</code> and fill in all required values

7. VIEWING RESULTS

```
```bash
```

```
 npm run test:report # HTML report with screenshots/traces
 npm test -- --reporter=json # Export JSON
```

```
```
```

Quick Reference: [Playwright Docs](<https://playwright.dev>) | See
`QUICK\_START.md` for fast start

8. FULL BACKEND TEST SUITE

Run from the link_jpmc/ root:

- *npm run test*
- *This runs both unit and integration tests in sequence:*
 - *npm run test:unit*
 - *npm run test:integration*

8.1 Unit Tests

- *Run from the link_jpmc/ root:*
- *npm run test:unit*
- *This runs unit tests under link_jpmc/test/unit/**.*

8.2 Integration Tests

- *Run from the link_jpmc/ root:*
- *npm run test:integration*
- *This runs integration tests under link_jpmc/test/integration/int_jpmc_*/**.*